

Intelligent Industrial Energy Monitoring System

Backend and Dashboard Step-by-Step Codex Build Plan

Scope	Dashboard backend, database, APIs, live updates, interface and deployment
Target	Raspberry Pi 4 deployment with a local development environment
Prepared for	Mosa Masedi - Postgraduate Diploma research project
Date	15 August 2026
Status	Implementation specification - Version 1.0

Purpose of this document

Give this PDF to Codex as the controlling specification. Ask Codex to implement one numbered step at a time, run the stated tests, and stop at every gate for your review. The fixed decisions and data contracts are intended to prevent the dashboard, backend and database from drifting apart.

Project basis: Introduction of IDD118G / IDD108G - Industrial Research Project, Prof. Chunling Du (2026), together with the approved project abstract.

How to use this specification

The document is written as a controlled handoff to a coding agent. It fixes the MVP scope, the technical stack, the meaning of each value, the API contracts and the tests that prove each part works.

Rule	Instruction
1. One step at a time	Tell Codex to implement only the next numbered step. Review its code and test output before authorising the following step.
2. No silent substitutions	Codex must not replace PostgreSQL, FastAPI, React, TypeScript or the stated routes without first explaining the trade-off and obtaining approval.
3. Keep the MVP safe	The first release is monitoring and advisory only. It must never operate a real contactor, relay or industrial load.
4. Show evidence	At each gate Codex must list changed files, commands run, test results, remaining risks and any assumption made.
5. Preserve student ownership	Understand, review and modify all code. Keep a design-decision log and AI-use declaration as required by the module policy.

Contents

- 1. Project alignment and fixed scope
- 2. Final system architecture and technology choices
- 3. Dashboard information architecture and visual requirements
- 4. Data model and engineering calculation rules
- 5. REST and WebSocket API contract
- 6. Repository structure and coding standards
- 7. Step-by-step implementation plan
- 8. Verification, deployment and demonstration
- 9. Copy-and-paste master prompt for Codex

Important academic-use note

The supplied module document permits AI as a supportive tool and requires transparent, limited use. It also states that AI must not replace the student's engineering judgement or generate complete submitted reports, designs, calculations or conclusions. Treat this plan as an implementation aid: validate every choice, write your own academic explanation, retain test evidence, and include the required AI Use Declaration.

1. Project alignment and fixed scope

The dashboard is one subsystem of the larger intelligent industrial energy monitoring project. This plan deliberately isolates that subsystem so it can be completed and tested before real Cortex-M4 data, machine-learning models or load switching are connected.

1.1 Module alignment

Requirement from supplied module brief	How this dashboard work will provide evidence
System-level design and integration	Documented data flow from source to API, database, calculations, live stream and user interface.
Engineering knowledge and tool selection	A justified stack, defined contracts, timestamps, units, validation rules and version-controlled migrations.
Testing and validation	Automated backend tests, frontend checks, data accuracy tests, failure tests and a repeatable demonstration script.
Professional communication	README, API documentation, database diagram, design-decision log and traceable acceptance evidence.
Judgement and responsibility	Read-only MVP, explicit stale-data behaviour, audit logging, secure configuration and no direct control of loads.

1.2 MVP goal

MVP outcome

A user opens a responsive web dashboard, sees live and historical energy measurements from a simulator, views calculated demand and energy KPIs, inspects forecast and anomaly placeholders, acknowledges alarms, and confirms system/device health. The same ingestion contract can later accept data from the Raspberry Pi gateway without rewriting the dashboard.

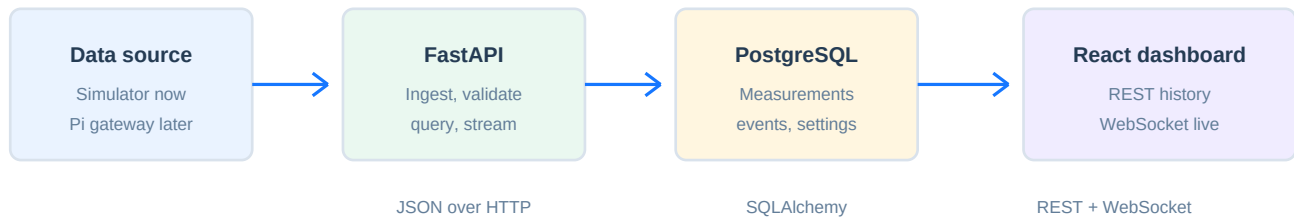
1.3 In scope

- FastAPI backend with validation, business rules, query services, authentication, structured logging and health endpoints.
- PostgreSQL database with migrations, seed data, indexes, retention-ready tables and audit history.
- React and TypeScript dashboard with Overview, Live, History, Forecast and Anomalies, Alarms, Devices and Settings pages.
- A deterministic simulator that creates realistic measurement, forecast, anomaly, alarm and connectivity data.
- REST APIs for history and configuration, plus WebSocket events for near-real-time updates.
- Docker Compose for development and Raspberry Pi 4 deployment, automated tests and user/developer documentation.

1.4 Explicitly out of scope for this build

- Cortex-M4 firmware, current or voltage sensing electronics, Modbus/RS485 drivers and calibration.
- Training of forecasting or anomaly-detection models. The dashboard will consume their future outputs through fixed tables and APIs.
- Real relay, contactor or load-control commands. Any future control page remains advisory or simulated until a separate safety review is complete.
- Cloud hosting, public internet exposure, billing-grade metering or production certification.

2. Final architecture and technology choices



All stored timestamps are UTC; the interface displays Africa/Johannesburg time.

2.1 Fixed technology stack

Layer	Selected technology	Reason for selection
Backend	Python, FastAPI, Pydantic	Matches the project's Python and ML ecosystem; automatic OpenAPI documentation; strong input validation.
Database	PostgreSQL with SQLAlchemy 2 and Alembic	Reliable relational storage, good time filtering and aggregation, migrations and Raspberry Pi ARM64 support.
Frontend	React, TypeScript, Vite	Component-based custom dashboard with type-safe API integration and fast development.
Data fetching	TanStack Query	Caching, retries, background refresh and clean server-state handling.
Charts	Recharts	Responsive trend, forecast, energy and comparison charts without building chart primitives manually.
Styling	Tailwind CSS	Consistent responsive design and status colours with a small visual system.
Live channel	Native FastAPI WebSocket	Low-complexity live updates while REST remains the source for historical queries.
Deployment	Docker Compose and Nginx	Reproducible setup, separation of services and a single browser entry point.
Testing	Pytest, HTTPX, Vitest, React Testing Library, Playwright	Unit, API, component and end-to-end coverage.

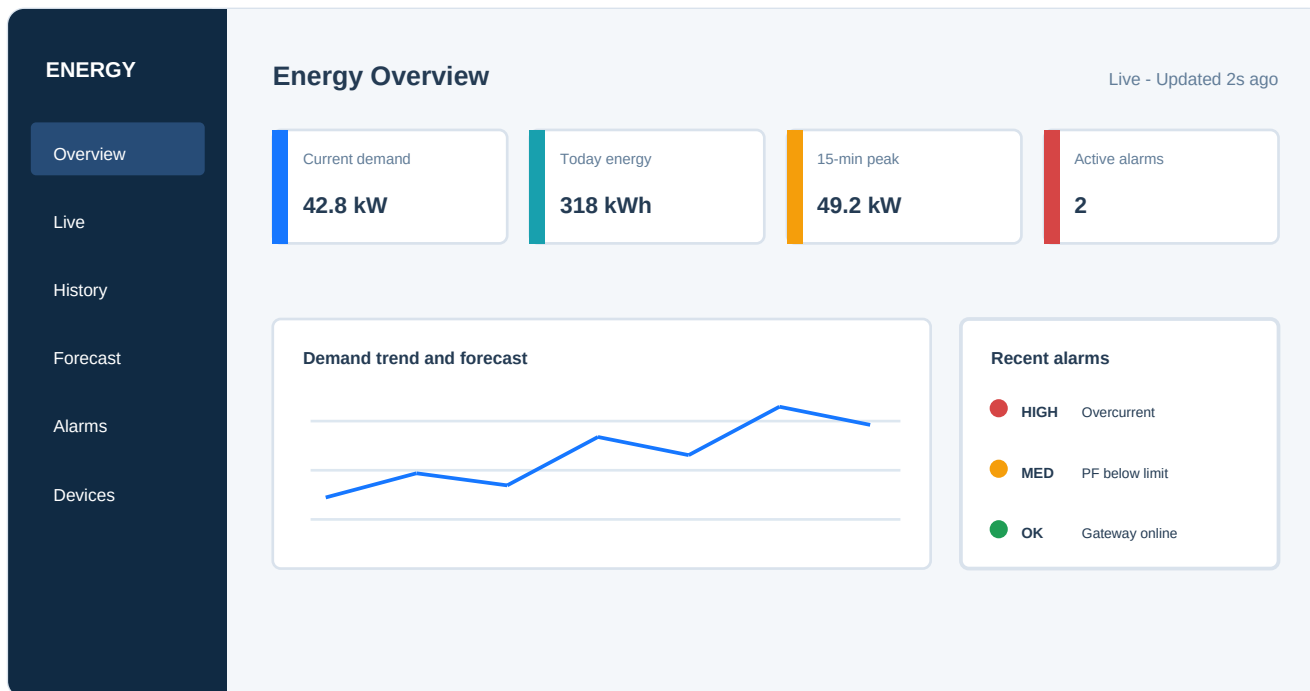
2.2 Architectural boundaries

- The frontend never queries PostgreSQL directly. All data passes through versioned backend APIs.
- The ingestion service writes validated records; the query service reads and aggregates them. These are separate modules even when deployed in one API container.
- REST is authoritative for page loads and history. WebSocket messages notify the UI of new values; after reconnect, the UI refetches REST data.
- ML services and the Pi gateway integrate later through the documented forecast, anomaly and measurement contracts.
- All measurement calculations that affect displayed totals live in backend services, not duplicated in React components.

2.3 Environment profiles

Profile	Data source	Purpose
development	Deterministic simulator	Local coding and repeatable tests.
demo	Simulator with scripted normal and abnormal scenarios	Project demonstrations and screenshots.
gateway	Authenticated HTTP ingestion from Raspberry Pi service	Later hardware integration.
test	Isolated test database	Automated tests; never uses development or production data.

3. Dashboard information architecture



3.1 Navigation and pages

Page	Required content	Primary user question
Overview	Four KPI cards; 24-hour demand trend; short-term forecast; recent alarms; device health; last update time.	What is happening now, and is anything wrong?
Live Monitoring	Per-device live voltage, current, kW, power factor and frequency; connection quality; 60-minute trend.	What is each monitored load doing right now?
Energy History	Date/device filters; power trend; daily energy bars; 15-minute demand; comparison to previous period; CSV export.	How was energy consumed over a selected period?
Forecast and Anomalies	Actual versus predicted demand; confidence band; model version; anomaly timeline; severity and explanation.	What is expected next, and what behaviour is unusual?
Alarms	Filterable alarm table; open, acknowledged and cleared states; details drawer; acknowledgement action.	Which conditions need attention and who handled them?
Devices	Device name, location, type, criticality, rated kW, last seen, status and data quality.	Are sources configured and communicating?
Settings	Demand threshold, stale timeout, tariff, timezone and simulator controls; admin-only edits.	Which values control dashboard interpretation?

3.2 Overview page - exact requirements

- KPI 1 Current demand: sum the latest non-stale active power values; show kW and trend versus 15 minutes earlier.
- KPI 2 Energy today: calculate each device's valid meter-counter delta from the counter nearest local midnight; show kWh, completeness and comparison to yesterday at the same elapsed time.
- KPI 3 Peak demand: maximum 15-minute rolling average since local midnight; show kW and time reached.
- KPI 4 Active alarms: count open plus acknowledged-but-not-cleared alarms; split by severity in the tooltip.
- Main chart: actual total kW for the last 24 hours, predicted kW for the configured horizon and a dashed demand threshold.
- Device status: online, stale, offline and disabled counts. Stale data must never look live.
- Every panel must include loading, empty, error and stale states; do not render fake zero values when data is missing.

3.3 Visual and interaction rules

Area	Rule
Layout	Desktop: fixed left navigation and 12-column content grid. Tablet: collapsible navigation. Mobile: stacked cards and horizontally scrollable tables.
Colours	Navy #102A43, blue #1677FF, teal #18A0AE, green #1F9D55, amber #F59E0B, red #D64545, light background #F4F7FA.
Status semantics	Green = normal/online; amber = warning/stale; red = critical/offline; blue = informational. Never use colour as the only signal - include icon and text.
Numbers	kW one decimal; kWh one decimal; voltage one decimal; current two decimals; PF two decimals; frequency two decimals. Display '-' for missing values.
Time	Store UTC. Display Africa/Johannesburg by default. Show absolute timestamp in tooltips and relative last-seen text beside status.
Charts	Readable axis titles with units; shared tooltip; no 3D effects; limit default view to useful points through backend aggregation.
Feedback	Toast on successful acknowledgement/settings update; inline message with retry on network failure; skeletons during initial load.
Accessibility	Keyboard navigation, visible focus, semantic labels, table headers, WCAG AA contrast and reduced-motion respect.

3.4 Frontend component inventory

Group	Components
Layout	AppShell, SideNav, TopBar, PageHeader, ConnectionBadge
Cards	KpiCard, StatusCard, DeviceSummaryCard
Charts	DemandTrendChart, EnergyBarChart, ForecastChart, PowerFactorChart
Tables	AlarmTable, DeviceTable, AnomalyTable, EventTable
Inputs	DateRangePicker, DeviceSelect, IntervalSelect, SeverityFilter, SettingsForm
States	LoadingPanel, EmptyState, ErrorPanel, StaleBanner, PermissionNotice

Control restriction

Do not create an enabled Start, Stop, Shed or Restore button in this dashboard build. A future control screen may display advisory recommendations, but real commands require a separate hardware, safety and authority specification.

4. Data model and calculation rules

4.1 Common data conventions

- Use UUIDs for user-facing entities and BIGSERIAL for high-volume measurement rows.
- Use TIMESTAMPTZ for every timestamp and serialize as ISO 8601 UTC, for example 2026-08-15T19:30:00Z.
- Use SI units in field names: voltage_v, current_a, active_power_kw, energy_kwh_total and frequency_hz.
- Use DECIMAL/NUMERIC for cumulative energy and tariff values; use double precision for sampled electrical values.
- Unique constraint on (device_id, measured_at) prevents duplicate measurement inserts. Duplicate ingestion returns accepted-but-duplicate, not a second row.
- Never use a numeric zero to represent missing data. Use NULL plus data_quality and a displayed '-' value.

4.2 Required tables

Table	Minimum fields	Purpose
users	id UUID PK; email unique; password_hash; role viewer/operator/admin; is_active; created_at; last_login_at	Authentication and accountability
devices	id UUID PK; code unique; name; location; phase_type; rated_power_kw; criticality; enabled; last_seen_at; created_at	Monitored load metadata
measurements	id BIGSERIAL PK; device_id FK; measured_at; voltage_v; current_a; active_power_kw; reactive_power_kvar; apparent_power_kva; power_factor; frequency_hz; energy_kwh_total; data_quality	Validated time-series measurements
forecasts	id BIGSERIAL PK; device_id nullable; generated_at; target_at; horizon_minutes; predicted_power_kw; lower_kw; upper_kw; model_version	Outputs from future ML service or simulator
anomalies	id UUID PK; device_id FK; detected_at; anomaly_type; severity; metric; actual_value; expected_value; score; explanation; status	Detected abnormal consumption
alarms	id UUID PK; device_id nullable; source; alarm_type; severity; message; triggered_at; acknowledged_at/by; cleared_at; status	Operational alarm lifecycle
system_events	id BIGSERIAL PK; event_at; actor_user_id nullable; event_type; entity_type; entity_id; details_json	Immutable audit and system history
settings	key PK; value_json; description; updated_at; updated_by	Demand limit, tariff, stale timeout, timezone and simulator options

4.3 Essential indexes

- measurements(device_id, measured_at DESC)
- measurements(measured_at DESC)
- forecasts(target_at, generated_at DESC)
- alarms(status, severity, triggered_at DESC)
- anomalies(status, detected_at DESC)
- system_events(event_at DESC)

4.4 Measurement validation

Field	Accepted range / rule	Behaviour outside rule
measured_at	Valid UTC timestamp; not more than 5 minutes in the future	Reject record with field-level 422 error
voltage_v	0 to 1,000 V	Reject; future hardware range can tighten limit
current_a	0 to configured 2,000 A	Reject above hard limit; warn above device rating
active_power_kw	-2,000 to 2,000 kW	Allow negative for future generation/export; dashboard labels direction
power_factor	0.00 to 1.00	Reject invalid value
frequency_hz	40.00 to 70.00 Hz	Accept as suspect only if explicit data_quality=suspect; otherwise reject
energy_kwh_total	Non-negative cumulative value	Accept reset only when reset flag/event is supplied; do not subtract across reset
data_quality	good, suspect or missing	Reject unknown enum

4.5 Calculation rules - implement on the backend

Metric	Exact rule
Device online	Latest record age <= online_timeout_seconds (default 10).
Device stale	Latest record age > 10 s and <= stale_timeout_seconds (default 60).
Device offline	No record, or latest record age > 60 s.
Current demand	Sum latest active_power_kw only for enabled, non-stale devices. Return excluded device count.
Energy today	For each device, use the last valid counter at/before local midnight as baseline when recent enough; otherwise use the first after midnight and mark the gap. Split at a reset and sum valid segments.
15-minute demand	Time-weighted mean active power over a rolling 15-minute window; not a single instantaneous maximum.
Peak demand	Maximum valid 15-minute demand value in the selected local day.
Previous comparison	Compare equal elapsed intervals, for example today 00:00-14:00 versus yesterday 00:00-14:00.
Estimated cost	energy_kwh x configured flat tariff for MVP; clearly label as estimate and store currency ZAR.
Data completeness	Expected samples versus valid samples for period, expressed as percentage. Expose gaps; do not interpolate silently.

Why the 15-minute rule matters

Peak demand must be calculated from a defined demand interval, not from the highest one-second power sample. Keep the interval configurable, record the selected value in Settings, and explain it in the project report using the relevant electricity tariff or test definition.

5. API contract

All endpoints use the prefix `/api/v1`. OpenAPI documentation must be available at `/docs` in development and protected or disabled for later production use. Lists use `page`, `page_size`, `total` and `items`. Errors use one standard envelope.

5.1 Endpoint inventory

Method	Route	Purpose
GET	<code>/health</code>	Process health; no database dependency
GET	<code>/ready</code>	Database connectivity and migrations ready
POST	<code>/auth/login</code>	Issue secure session/JWT; rate limited
POST	<code>/auth/logout</code>	Invalidate session
GET	<code>/auth/me</code>	Current user and role
POST	<code>/ingest/measurements</code>	Authenticated gateway/simulator batch ingest
GET	<code>/dashboard/summary</code>	KPIs, alarm counts and device status
GET	<code>/measurements/latest</code>	Latest values for one or all devices
GET	<code>/measurements/history</code>	Filtered and aggregated measurement series
GET	<code>/energy/daily</code>	Daily kWh, peak kW and completeness
GET	<code>/forecasts/latest</code>	Latest forecast curve for a selected horizon
GET	<code>/anomalies</code>	Filterable anomaly list
GET	<code>/alarms</code>	Filterable paginated alarm list
PATCH	<code>/alarms/{id}/acknowledge</code>	Acknowledge with user, time and note
GET	<code>/devices</code>	Device list and last-seen status
POST/PATCH	<code>/devices</code>	Admin create/update, never hard-delete data
GET/PATCH	<code>/settings</code>	Read settings; admin update with audit event
GET	<code>/events</code>	Audit and system events
WS	<code>/ws/live</code>	Live measurement, alarm and health events

5.2 Required query parameters

- History: `device_id` (repeatable), `from`, `to`, interval in `raw|10s|1m|15m|1h|1d`, `metric`, `timezone`.
- Daily energy: `device_id` optional, `from_date`, `to_date` and `timezone`.
- Alarm/anomaly list: `status`, `severity`, `device_id`, `from`, `to`, `page` and `page_size`.
- Backend limits: history range maximum 31 days per request; raw data maximum 24 hours; `page_size` maximum 200.

5.3 Measurement ingestion contract

```
POST /api/v1/ingest/measurements
Headers:
  Content-Type: application/json
  X-Ingest-Key:

{
  "source": "simulator",
  "records": [
    {
      "device_code": "LOAD-001",
      "measured_at": "2026-08-15T19:30:00Z",
      "voltage_v": 231.4,
      "current_a": 12.63,
      "active_power_kw": 2.71,
      "reactive_power_kvar": 0.74,
      "apparent_power_kva": 2.81,
      "power_factor": 0.96,
      "frequency_hz": 49.99,
      "energy_kwh_total": 1842.376,
      "data_quality": "good"
    }
  ]
}
```

Response: HTTP 202 with `accepted_count`, `duplicate_count`, `rejected_count` and an `errors` array containing `record_index`, `field` and `reason`. Reject the whole batch only for invalid authentication or malformed JSON; otherwise report per-record outcomes.

5.4 Dashboard summary response

```
{
  "generated_at": "2026-08-15T19:30:02Z",
  "timezone": "Africa/Johannesburg",
  "current_demand_kw": 42.8,
  "current_demand_change_pct": 4.2,
  "energy_today_kwh": 318.4,
  "energy_vs_previous_pct": -3.8,
  "peak_demand_kw": 49.2,
  "peak_demand_at": "2026-08-15T14:15:00Z",
  "demand_limit_kw": 50.0,
  "active_alarms": {"critical": 0, "high": 1, "medium": 1, "low": 0},
  "devices": {"online": 4, "stale": 1, "offline": 0, "disabled": 1},
  "data_completeness_pct": 99.3,
  "excluded_stale_devices": 1
}
```

5.5 Standard error envelope

```
{
  "error": {
    "code": "INVALID_DATE_RANGE",
    "message": "The end timestamp must be after the start timestamp.",
    "details": {"field": "to"},
    "request_id": "01J5..."
  }
}
```

5.6 Live WebSocket protocol

- Client connects after authentication and sends subscriptions: device IDs plus event types.
- Server sends `measurement.new` at a maximum of one event per device per second, `alarm.changed`, `device.status` and `heartbeat`.
- Every message contains `type`, `emitted_at`, `sequence` and `payload`. Sequence supports gap detection.
- Client shows Reconnecting after 5 seconds without heartbeat, retries with exponential backoff capped at 30 seconds, then refetches REST state after reconnect.
- WebSocket data is not the permanent source of truth. It updates the screen; PostgreSQL and REST provide recovery and history.

```
{
  "type": "measurement.new",
  "emitted_at": "2026-08-15T19:30:02Z",
  "sequence": 10482,
  "payload": {
    "device_id": "d5d3...",
    "device_code": "LOAD-001",
    "measured_at": "2026-08-15T19:30:02Z",
    "active_power_kw": 2.73,
    "voltage_v": 231.1,
    "current_a": 12.71,
    "power_factor": 0.96,
    "frequency_hz": 50.01,
    "data_quality": "good"
  }
}
```

5.7 Permissions

Action	Viewer	Operator	Admin
View dashboards and data	Yes	Yes	Yes
Export CSV	Yes	Yes	Yes
Acknowledge alarms	No	Yes	Yes
Edit devices/settings	No	No	Yes
Send load-control command	Not implemented	Not implemented	Not implemented

Security minimum: hash passwords with Argon2 or bcrypt; never commit secrets; use an `HttpOnly SameSite` cookie or short-lived access token plus refresh strategy; protect write routes with role checks; rate-limit login and ingestion; add request IDs and immutable audit events for every change.

6. Repository structure and coding standards

```

energy-dashboard/
  apps/
    api/
      app/
        api/v1/          # route modules only
        core/            # config, auth, logging, errors
        db/              # session, base, migrations helpers
        models/          # SQLAlchemy models
        schemas/         # Pydantic request/response schemas
        services/        # calculations and business logic
        repositories/    # database queries
        websocket/       # connection manager and events
        simulator/       # deterministic demo generator
      main.py
    alembic/
    tests/
    pyproject.toml
  web/
    src/
      api/               # generated/manual typed client
      components/
      features/          # overview, history, alarms, devices
      hooks/
      layouts/
      pages/
      routes/
      types/
      utils/
    tests/
    package.json
  infra/
    docker-compose.yml
    nginx.conf
  docs/
    architecture.md
    data-dictionary.md
    api-contract.md
    decisions/           # ADR-001, ADR-002, ...
    test-evidence/
  .env.example
  .gitignore
  Makefile
  README.md

```

6.1 Coding rules

- Keep API route handlers thin: validate input, call a service, return a schema. Do not place SQL or calculation formulas in route files.
- Use repository functions for database access and service functions for calculations. Write a focused unit test for every calculation rule.
- Generate TypeScript API types from OpenAPI where practical; otherwise keep one central typed client. Do not duplicate response shapes in page components.
- Use UTC-aware datetimes only. Reject naive timestamps in backend tests.
- Use linting and formatting: Ruff and type checking for Python; ESLint and Prettier for TypeScript. Fail CI on lint or test failures.
- No default production passwords, secrets in source, hard-coded localhost URLs or hard-coded device IDs.
- Each architectural change requires a short ADR recording context, options, decision and consequences.

6.2 Required environment variables

```
APP_ENV=development
DATABASE_URL=postgresql+psycopg://energy:db:5432/energy
APP_SECRET_KEY=
INGEST_API_KEY=
DEFAULT_TIMEZONE=Africa/Johannesburg
ONLINE_TIMEOUT_SECONDS=10
STALE_TIMEOUT_SECONDS=60
DEMAND_INTERVAL_MINUTES=15
DEMAND_LIMIT_Kw=50
TARIFF_ZAR_PER_KWH=3.00
SIMULATOR_ENABLED=true
SIMULATOR_SEED=118
VITE_API_BASE_URL=/api/v1
VITE_WS_URL=/api/v1/ws/live
```

7. Step-by-step implementation plan

Codex should execute the following sequence. Each step has a gate. If a gate fails, Codex fixes that step before moving on. The user should commit a working checkpoint after every gate.

STEP 1: Freeze scope and create the engineering records

Objective: Turn this specification into traceable project requirements before writing application code.

Codex must:

- Create docs/requirements.md with numbered requirements FR-001 onward and NFR-001 onward.
- Create docs/decisions/ADR-001-stack.md, ADR-002-time-and-units.md and ADR-003-monitoring-only-safety-boundary.md.
- Create docs/acceptance-matrix.md mapping every requirement to a verification test.
- Create the repository tree, .gitignore, README skeleton and .env.example with placeholders only.

Gate - do not continue until:

- Every in-scope feature and out-of-scope item is written down.
- No secret value exists in the repository.
- The user has reviewed and accepted the stack and monitoring-only boundary.

STEP 2: Scaffold the backend, frontend and local containers

Objective: Produce a minimal full-stack system that starts reliably before adding domain logic.

Codex must:

- Create FastAPI app with /health, structured JSON logging, request ID middleware and central error handling.
- Create React/TypeScript/Vite app with router, blank AppShell and a typed health check.
- Create Docker Compose services for PostgreSQL, API, web dev server and optional Nginx profile.
- Add Make targets or documented commands for up, down, test, lint, migrate and seed.

Gate - do not continue until:

- docker compose up --build starts without manual edits.
- GET /health returns 200 and the browser shows Backend connected.
- Backend and frontend lint commands pass.

STEP 3: Implement configuration, database sessions and migrations

Objective: Create a versioned database foundation with a readiness check.

Codex must:

- Implement typed environment configuration and startup validation.
- Create SQLAlchemy models for users, devices, measurements, forecasts, anomalies, alarms, system_events and settings.
- Create Alembic migration 0001 with foreign keys, checks, unique constraints and required indexes.
- Implement /ready that fails clearly when the database or migrations are unavailable.

Gate - do not continue until:

- A clean database migrates from zero to head and downgrades one revision in development.
- Database constraints reject duplicate measurements and invalid enums.
- /ready reports database-ready without exposing credentials.

STEP 4: Create seed data and the deterministic simulator

Objective: Make the dashboard fully demonstrable before hardware exists.

Codex must:

- Seed at least five devices: compressor, conveyor motor, pump, heater and lighting load, with different rated powers and criticalities.
- Generate one-second measurements using a fixed seed and time-of-day load profiles; maintain monotonic cumulative energy.
- Create scriptable scenarios: normal operation, peak demand, low power factor, sudden overconsumption, device dropout and meter reset.
- Generate forecast and anomaly records through the same schemas later ML services will use.

Gate - do not continue until:

- Running the same seed and time range produces identical records.
- All scenarios are selectable and documented.
- No simulator-only field leaks into the public measurement contract.

STEP 5: Build secure measurement ingestion

Objective: Accept future gateway data through one validated, idempotent interface.

Codex must:

- Implement POST /ingest/measurements with X-Ingest-Key authentication and batch limits.
- Validate timestamps, device codes, units, ranges, data quality and energy resets.
- Insert valid records efficiently, detect duplicates and report per-record errors.
- Update device last_seen_at and write relevant system events without blocking the batch unnecessarily.

Gate - do not continue until:

- Contract tests cover valid, duplicate, partial-invalid, unknown-device, future-timestamp and unauthorised cases.
- One bad record cannot silently corrupt or mislabel another record.
- A 60-record batch completes within the locally recorded performance target.

STEP 6: Implement the calculation and query services

Objective: Create one tested source of truth for every displayed metric.

Codex must:

- Implement device status, current demand, energy today, equal-period comparison, 15-minute demand, peak demand, completeness and estimated cost.
- Implement aggregation by raw, 10-second, 1-minute, 15-minute, hourly and daily intervals.
- Return bucket timestamps, sample_count and completeness so the UI can show gaps.
- Create unit tests with hand-calculated fixtures, stale-device cases, missing samples and a cumulative-meter reset.

Gate - do not continue until:

- Every calculation matches the stated fixture result within documented tolerance.
- Stale or missing devices are excluded and reported, never converted to zero.
- History limits prevent unbounded raw queries.

STEP 7: Expose the read APIs and OpenAPI contract

Objective: Provide all REST data needed by the dashboard before implementing screens.

Codex must:

- Implement summary, latest measurements, history, daily energy, forecasts, anomalies, alarms, devices, settings and events endpoints.
- Apply standard pagination, filtering, response schemas and error envelopes.
- Add OpenAPI examples for the main success and error responses.
- Export/openapi.json and use it to create or verify frontend TypeScript types.

Gate - do not continue until:

- API integration tests pass against an isolated database.
- The OpenAPI document validates and no route returns untyped dictionaries.
- History of 24 hours at one-minute resolution responds within 2 seconds on the target Pi or documented equivalent test machine.

STEP 8: Implement authentication, roles and audit events

Objective: Protect viewing and write actions without hiding engineering errors.

Codex must:

- Create login, logout and current-user endpoints with securely hashed passwords and expiry.
- Implement viewer, operator and admin permission dependencies.
- Create seeded users only through explicit setup commands; force credentials to come from environment or interactive setup.
- Record login failures, alarm acknowledgements and configuration changes without logging passwords or tokens.

Gate - do not continue until:

- Role tests prove viewers cannot acknowledge alarms and operators cannot edit settings.
- Tokens/cookies expire and unauthenticated API calls return 401 consistently.
- Application logs contain no secret or password values.

STEP 9: Implement WebSocket live updates and recovery

Objective: Add live behaviour without making the UI dependent on a perfect connection.

Codex must:

- Create a connection manager, subscription message, heartbeat and sequence number.
- Publish measurement.new, alarm.changed and device.status events after committed database changes.
- Add client reconnect with exponential backoff, a visible connection state and REST refetch after recovery.
- Throttle/coalesce updates to at most one event per device per second for the browser.

Gate - do not continue until:

- Live values update within 2 seconds during the simulator run.
- Stopping and restarting the API produces Reconnecting, then recovery without a page refresh.
- A forced sequence gap triggers REST refetch and correct current values.

STEP 10: Build the dashboard shell and design system

Objective: Create the stable visual foundation before adding data-heavy pages.

Codex must:

- Implement AppShell, navigation, protected routes, TopBar, connection badge and user menu.
- Define the documented colour tokens, typography, spacing, cards, tables, buttons and status badges.
- Create reusable loading, empty, error, permission and stale-data states.
- Verify desktop, tablet and mobile navigation and keyboard focus order.

Gate - do not continue until:

- All seven pages are routable with placeholder content and correct active navigation.
- No horizontal page overflow at 375 px width except inside deliberately scrollable tables.
- Automated accessibility smoke checks find no serious violations.

STEP 11: Build the Overview and Live Monitoring pages

Objective: Deliver the main demonstration path with correct values and honest data status.

Codex must:

- Build the four exact KPI cards and show calculation timestamp, unit and comparison context.
- Build the 24-hour actual/forecast/threshold chart, recent alarms list and device status summary.
- Build the Live page with device cards/table and 60-minute trend; update via WebSocket.
- Show stale banners and exclude stale devices exactly as the summary API reports.

Gate - do not continue until:

- KPI values equal the API responses and test fixture calculations.
- No component performs an alternative energy or peak-demand calculation.
- Loading, no-data, API-error, partial-data and stale states are visually verified.

STEP 12: Build History, Forecast and Anomaly pages

Objective: Support engineering analysis rather than only a live display.

Codex must:

- Implement URL-backed date, device and interval filters so a view is reproducible and shareable.
- Render power history, daily energy and 15-minute demand charts using backend buckets and completeness flags.
- Render actual versus forecast with confidence band and model version; show 'model data unavailable' instead of inventing a prediction.
- Render anomaly list/timeline with type, metric, expected, actual, score, severity and explanation.
- Export the currently filtered historical dataset as CSV with UTC timestamp and explicit units in headings.

Gate - do not continue until:

- Changing a filter changes the API query and browser URL.
- CSV values match displayed/filter data and open correctly in common spreadsheet software.
- Large time ranges use aggregation and remain responsive.

STEP 13: Build Alarms, Devices and Settings pages

Objective: Complete operational visibility and controlled configuration.

Codex must:

- Build filterable/paginated alarm table, details drawer and operator/admin acknowledgement form with note.
- Build device status table and admin create/edit form; use disable rather than hard delete.
- Build settings form for demand limit, demand interval, tariff, timezone, timeouts and simulator profile.
- Use optimistic UI only when rollback and error display are implemented; otherwise refetch after confirmed writes.

Gate - do not continue until:

- Acknowledgement records current user/time and appears in audit events.
- Invalid settings are rejected consistently by frontend and backend.
- Non-admin users cannot see active edit controls or bypass permissions through direct API calls.

STEP 14: Complete automated and failure testing

Objective: Prove correctness, resilience and usability against the acceptance matrix.

Codex must:

- Run backend unit and integration tests, frontend component tests and Playwright end-to-end flows.
- Test database unavailable, API unavailable, WebSocket interrupted, stale device, missing interval, duplicate records and invalid auth.
- Measure key response times, ingest throughput, page load and memory/CPU on Raspberry Pi or comparable constrained hardware.
- Store dated test outputs and screenshots under docs/test-evidence; document defects and retests.

Gate - do not continue until:

- All mandatory acceptance tests pass; no critical/high defect remains open.
- Backend service/calculation coverage is at least 80 percent, while critical formulas and permissions are fully covered.
- Results are repeatable from documented commands on a clean checkout.

STEP 15: Package for Raspberry Pi 4 and operate safely

Objective: Create a repeatable LAN-only deployment with backup and recovery instructions.

Codex must:

- Use ARM64-compatible container images and multi-stage builds; serve the built React app through Nginx.
- Add health checks, restart policies, named database volume and bounded logs.
- Document initial setup, migrations, admin creation, backup, restore, upgrade and rollback.
- Bind to the intended LAN interface; do not expose database or API internals directly to the public internet.

Gate - do not continue until:

- A clean Pi starts the system after documented commands and survives a reboot.
- A database backup restores into a clean volume and the dashboard shows the restored records.
- Power interruption testing produces no duplicate/corrupt measurement rows beyond documented behaviour.

STEP 16: Prepare the demonstration and engineering handoff

Objective: Make the implemented evidence understandable, reproducible and suitable for prototype evaluation.

Codex must:

- Complete README, architecture, data dictionary, API contract, decisions, test summary and limitations.
- Prepare a 7-10 minute demonstration: normal load, peak, anomaly, dropout, acknowledgement and reconnect.
- Record final requirements traceability and list what remains for gateway, ML and load-control integration.
- Add an AI Use Declaration consistent with the institution's requirements and describe personal verification performed.

Gate - do not continue until:

- A new reader can run the project without undocumented steps.
- The demo works from a fresh seed and every displayed value can be traced to source data and a documented rule.
- The user can explain each design choice, formula, test and limitation without relying on generated text.

8. Verification and acceptance

The following tests are the minimum Definition of Done for the dashboard subsystem. Values marked 'initial target' should be measured and revised with evidence if the Raspberry Pi baseline justifies a different threshold.

Area	Verification	Pass criterion
Functional	Overview values match API and hand-calculated fixtures	100 percent of mandatory fixtures pass
Ingestion	Valid, duplicate and invalid records handled correctly	No silent loss or duplicate row
Live update	Simulator measurement to visible dashboard update	≤ 2 s typical on LAN
History	24 hours at 1-minute resolution	API ≤ 2 s; chart interaction remains smooth
Data honesty	Stale, offline, missing and partial data	Correct explicit state; never displayed as live zero
Demand	15-minute rolling average and daily peak	Matches controlled fixture within numeric tolerance
Security	Authentication, roles, rate limit and secret scan	All role-denial tests pass; no committed secret
Recovery	API/WebSocket/database restart	UI informs user and recovers without corrupted data
Accessibility	Keyboard, labels, focus and contrast	No serious automated violation; manual keyboard pass
Compatibility	Desktop and 375 px mobile viewport	All core actions usable; no clipped content
Deployment	Fresh ARM64/Pi installation and reboot	One documented setup; services return healthy
Documentation	Trace requirement to design and test evidence	No mandatory requirement without evidence

8.1 Demonstration dataset

- Normal: stable motor and heater profiles, no active alarms.
- Peak: combined loads cross 90 percent, then 100 percent of the configured demand limit; warning and high alarm states appear.
- Abnormal: one load rises 35 percent above expected profile; anomaly record includes expected, actual, score and explanation.
- Power quality: power factor falls below configured warning level.
- Communication: one device stops sending; online becomes stale then offline without falsely reducing historical energy.
- Recovery: API restarts; WebSocket reconnects; current REST state is refetched.

8.2 Evidence to retain

Git commits; requirements matrix; ADRs; migration files; OpenAPI file; unit/API/E2E test output; simulator seed and scenario; screenshots; response-time table; Pi CPU/RAM observations; defect/retest log; backup/restore evidence; and signed/dated AI-use declaration.

9. Copy-and-paste master prompt for Codex

Attach this PDF to Codex and paste the following prompt. For later messages, replace STEP_NUMBER with the next authorised step.

```
You are implementing the dashboard subsystem for my postgraduate project,
"Intelligent Industrial Energy Monitoring and Predictive Load Management
System". The attached PDF is the controlling technical specification.

Work only on STEP_NUMBER from Section 7. Do not implement later steps.

Rules:
1. Read the entire attached specification before changing files.
2. Inspect the existing repository and preserve unrelated user changes.
3. Follow the fixed stack, repository boundaries, field names, units, API
   routes, calculation rules, security roles and monitoring-only safety scope.
4. If the repository conflicts with the PDF, stop and explain the conflict.
5. Do not enable any real relay, contactor, load-shedding or restoration action.
6. Do not invent sensor, ML or gateway results. Use the deterministic simulator
   and clearly label simulated data.
7. Implement production-quality code for this step, including migrations,
   validation, errors, logs, tests and documentation that belong to the step.
8. Run the gate checks stated for this step. Fix failures before reporting.
9. Do not silently change a contract. Propose a documented ADR and wait for my
   approval if a contract must change.
10. Keep all secrets out of source and provide placeholders in .env.example.

At the end, report:
- outcome;
- files added or changed;
- commands run and exact test results;
- how each gate was satisfied;
- assumptions and risks;
- the single recommended next step.

Do not proceed beyond STEP_NUMBER until I approve it.
```

9.1 Review questions to ask Codex after every step

- Show me where each requirement in this step is implemented and tested.
- Which calculation or engineering assumption did you make, and where is it documented?
- What can fail in this implementation, and what will the user see when it fails?
- Which test would catch a regression in the most critical behaviour?
- Did you modify any API field, unit, route, security boundary or database meaning from the PDF?
- Explain the code in beginner-friendly terms so I can verify and defend it.

Recommended first instruction

Start with Step 1 only. Do not ask Codex to build all 16 steps in one request. A one-shot build is much harder to inspect, test and defend as your own engineering work.

10. Final handoff checklist

Area	Completion condition	Done
Scope	Monitoring/dashboard only; no real control path	☐
Architecture	Frontend -> versioned API -> services -> repositories -> PostgreSQL	☐
Data	UTC storage, explicit units, null for missing, duplicates prevented	☐
Metrics	Current demand, energy, 15-minute peak and completeness match fixtures	☐
Live	WebSocket reconnects and REST restores state	☐
UX	Loading, empty, error, stale and partial-data states implemented	☐
Security	Roles, secret handling, audit events and login protection tested	☐
Testing	Unit, API, component and E2E gates pass	☐
Pi	ARM64 deployment, reboot, backup and restore verified	☐
Evidence	ADRs, traceability, test evidence and AI declaration retained	☐

Definition of success

The subsystem is successful when a clean Raspberry Pi deployment can accept deterministic simulated measurements, store and calculate them correctly, display live and historical information with honest data-quality states, survive connection failure, enforce permissions, and reproduce the full demonstration from documented commands.

Source used

Du, C. (2026) *Introduction of IDD118G / IDD108G: Industrial Research Project - Focus: System Design, Engineering Decision-Making, and ECSA Alignment*. Course presentation supplied by the student. Relevant project requirements include hardware/software integration, system-level design, tool justification, implementation planning, testing and evaluation, professional reporting and responsible AI use.

End of specification